DECAF20 Parser Submission 2

Christopher Harris
CS 4308 Concepts of Programming Languages – Section 01
College of Computing and Software Engineering
Kennesaw State University
Marietta, Georgia
charr396@students.kennesaw.edu
October 27, 2026

I. Problem

The DECAF20 scanner iterates through a given DECAF20 file to collect each token and gather its key attributes, such as the token type and line number. However, its function is limited to data gathering, as it treats each one individually and does not take into account the order in which they appear. Therefore, the tokens could be in a scrambled, nonsensical order, which would still not impact the results of the scanner list. This is why a parser is needed. The parser needs to go through the token list in order, paying special attention to order and correct grammatical syntax to ensure it is proper and valid. By verifying the code in this manner, the parser can confirm its validity or throw syntax errors, something the scanner does not perform.

II. Purpose and Summary

The purpose of this deliverable was to develop a Python parser for the DECAF20 programming language, utilizing the results from the scanner. An implementation approach similar to the scanner's was employed, breaking down the parsing of program file sections into separate functions while looping through the token list. This lined up each program section with a specialized method to handle it. A Parser class was created to not only store scanner results, including the token list and source lines, but also the current list position and any error tokens that may arise. The class includes functions suited for different operations: parsing functions, token functions, and error functions.

The parsing functions include methods such as parse_declaration(), parse_function(), parse_block(), etc. These correspond with each possible section included in a DECAF20 program, handling variable declarations, function declarations, function bodies, if/else statements, for/while loops, and expressions. Each function expects certain criteria from each section to ensure validity and proper syntax throughout.

The token functions include current_token(), advance_token(), match_token(), etc. These deal with the actual token list navigation and validation. This involves actions to view the current and upcoming tokens to consume them in order to proceed in the program. They also match tokens up with their expected type based on what the parsing functions are expecting to guarantee proper syntax.

```
PS C:\Users\chris\PycharmProjects\DECAF20-scanner\src> python parser.py tests/bad1.decaf
*** Error line 1.
void foo(int) {
              ^
*** syntax error
```

Fig. 1. Test Case Output

Lastly, the error functions include syntax_error() and report_syntax_error(). These methods raise errors when they are found within the parsing functions. All relevant information for each error, such as the line and column numbers, is gathered and then used to properly format print statements for console reporting, communicating exactly where the error occurred and what kind of error it was.

III. Test Input and Output Results

The screenshot in Fig. 1 demonstrates the executed parser output of one test file, whose DECAF20 source code is shown below. In the output, the parser was able to identify and throw the syntax error caused by the missing variable name at the pinpointed location.

```
void foo(int) {

}
```

IV. Conclusion

The built parser completes the second half of the pipeline needed to process the DECAF20 language. First, a program file is scanned for all its tokens and compiled into a list with their key attributes. Then, the tokens are parsed, validating the correctness of the syntax only while still ignoring semantic value. The parser is able to take in any list of tokens, not just those created by the scanner.

Building the parser showed the importance of error catching and enforcing it, allowing the transition and transformation between gathering token data and analyzing it to be seen. The next step will be to integrate the creation of a parse tree within the parser in order to give a full view of the parsing process for each program file. This will be the focus of the final deliverable of the project.

REFERENCES

[1] M. Alghoul, "Building a Custom Parser: A Step-by-Step Guide," *Medium*, Aug. 5, 2023. [Online]. Available: https://medium.com/@menta8seca/building-a-custom-parser-a-step-by-step-guide-b0a92340ff51

[2] R. W. Sebesta, *Concepts of Programming Languages*, 12th ed. Pearson, 2019.